ENTORNOS DE DESARROLLO
UD1: Fases del desarrollo de una aplicación

Proceso de desarrollo

- Ciclo de vida: Diferentes etapas que surgen desde que se produce la necesidad de crear una aplicación hasta que finalmente se implanta.

- La realidad en este tema siempre supera la ficción!

Enlace: Developer´s life

Fases del desarrollo software

- Análisis
- Diseño
- Implementación
- Pruebas
- Documentación
- Explotación
- Mantenimiento

Análisis

Fases del desarrollo software.
Análisis

Es una de las etapas más importantes ya que hay que comprender completamente el problema a resolver y su contexto.

Esto se traduce en un conjunto de **resquisitos** hechos por el usuario, indicando las distintas funcionalidades que desea que el sistema tenga.

Es una etapa crítica en la que **hay que invertir el tiempo que sea necesario** y en la que se genera un documento de requerimientos.

¿Qué es un requisito?

- Puede verse como una declaración abstracta de alto nivel de un servicio que el sistema debe proporcionar.

- Como tal, son el contrato en sí mismo. Cuanto menos detallado, y más abierto estén peor, más cosas habrá que desarrollar por estar sujeto a interpretación.

Clasificación (I)

- **Requisitos de usuario:**
  - Declaraciones en lenguaje natural y en diversos diagramas de los servicios del sistema y de las restricciones bajo las que debe operar.
- **Requisitos de sistema:**
  - Un documento estructurado que determina las descripciones detalladas de los servicios de sistema.
  - Escrito como contrato entre el cliente y los que desarrollan en proyecto.
  - Deben ser una especificación completa y consistente del sistema.

Clasificación (I)

Un requisito de usuario

1.- El sistema debe permitir representar y acceder a archivos externos creados por otras herramientas

Requisitos del sistema asociados

1.- El usuario deberá poder definir el tipo de un nuevo archivo externo.

2.- Cada tipo de archivo tendrá una herramienta asociada, que se aplicará al archivo.

3.- Cada tipo de archivo se representará con un icono específico.

4.- El usuario deberá poder definir el icono que representa un tipo de archivo externo.

5.- Cuando el usuario selecciona un icono que representa un archivo externo, el efecto es aplicar la herramienta asociada con este tipo de archivo al archivo representado por el icono seleccionado.

Clasificación (II)

- **<u>Requisitos funcionales</u> (RF):**
  - Definición de los servicios que el sistema debe proporcionar, cómo debe reaccionar a una entrada particular y cómo se debe comportar ante situaciones particulares.
  - Ejs.:

1. Se deben poder realizar búsquedas en base a diferentes criterios.
2. Se deben proporcionar diferentes visores para que el usuario lea los documentos recuperados.
3. Cada factura tendrá un número único y correlativo y la fecha.

Clasificación (II)

- **<u>Requisitos no funcionales</u> (RNF):**
  - Restricciones que afectan a los servicios o funciones del sistema, tales como restricciones de tiempo, sobre el proceso de desarrollo, estándares, etc.
  - Que un RF no se cumpla, hace que el sistema se degrade. Pero, **que un RNF no se cumpla, puede implicar la inutilización del sistema.**

Clasificación (II)

Dentro de los RNF se distinguen 3 tipos:

- <u>Requisitos del producto:</u> eseticos o de arquitectura(equipos que usamos)
  - Especifican el comportamiento del producto obtenido: velocidad de ejecución, memoria requerida, porcentaje de fallos aceptables (SLA), …
- <u>Requisitos organizacionales:</u> Manual de usuario, configuracion de instalacion, etc..)
  - Son una consecuencia de las políticas y procedimientos existentes en la organización: procesos estándar utilizados, de fechas de entrega, documentación a entregar, …
- <u>Requisitos externos:</u> Si necesitamos un API de 3, hay que darse de alta, pagar en su caso, etc.
  - Presentan factores externos al sistema y a su proceso de desarrollo: interoperabilidad del sistema con otros, requisitos legales, éticos, …

Clasificación (II)

- Importante, que los RNF estén reflejados con exactitud.

Ej:

1. RNF imprecisos (una primera versión)
   - Los usuarios especializados deberán utilizar el sistema fácilmente.
   - El sistema deberá estar organizado para minimizar los errores del usuario.
2. RNF verificables (detallados)
   - Los usuarios experimentados deberán poder utilizar todas las funciones del sistema después de un total de dos horas de entrenamiento.
   - Después de este entrenamiento, el número medio de errores cometidos por los usuarios experimentados no excederá de dos por día.

Es muy tipico, se hace habitualmente, normalmente no se les explica como va el programa, asi ven, si generan muchos errores, etc, luego se hace una encuesta

Características

- **<u>En general:</u>**
  - Claros y concretos, evitando imprecisiones y ambigüedades
  - Concisos, sin rodeos ni figuras retóricas
  - Completos y consistentes
- **<u>RF:</u>**
  - Deben estar redactados de tal forma que sean comprensibles para usuarios sin conocimientos técnicos avanzados (de Informática, se entiende).
  - Deben especificar el comportamiento externo del sistema y evitar, en la medida de lo posible, establecer características de su diseño.
  - Deben priorizarse (al menos, se ha de distinguir entre requisitos obligatorios y requisitos deseables)
- **<u>RNF</u>**
  - Han de especificarse cuantitativamente, siempre que sea posible (para que se pueda verificar su cumplimiento).

Diseño

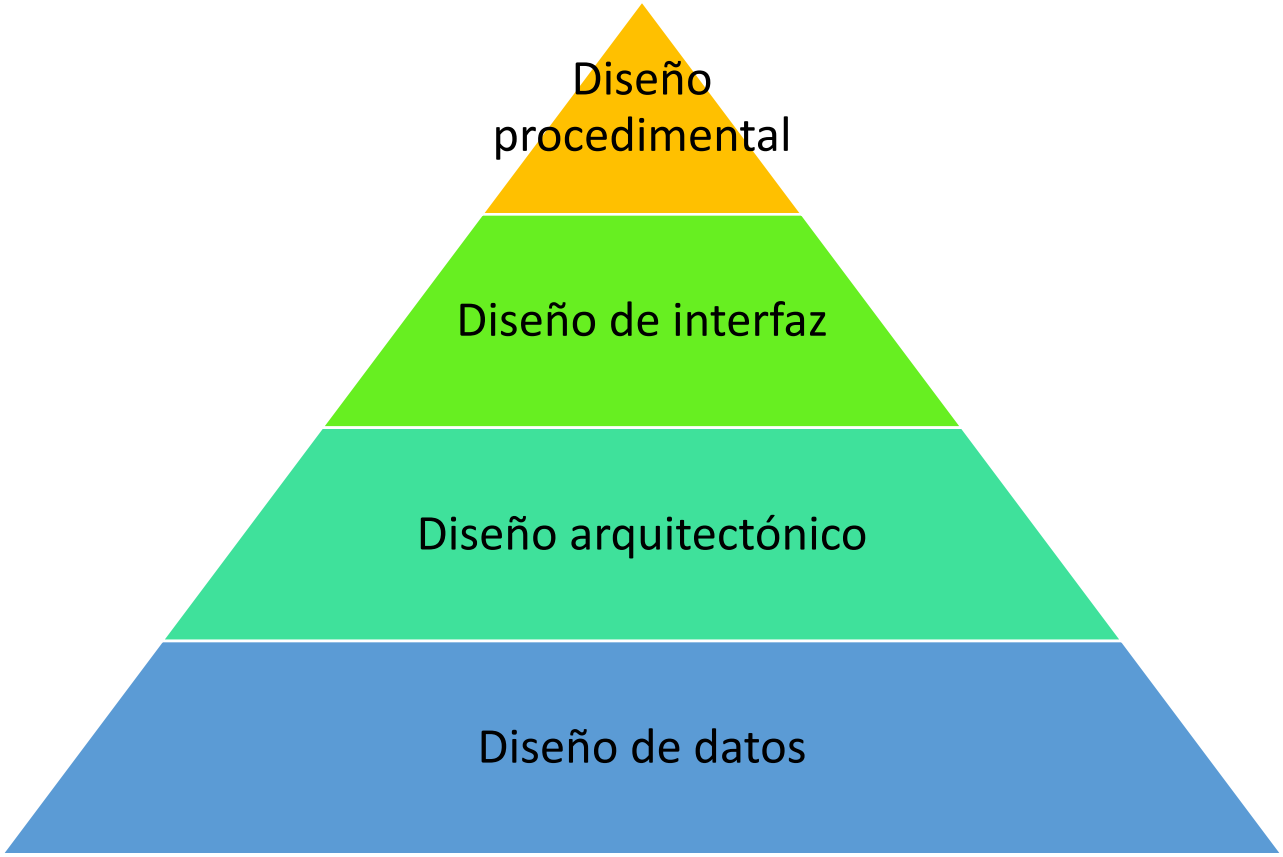
Fases del desarrollo software.
Diseño

Una vez identificados los requisitos es necesario componer la forma en que se solucionará el problema.

Traduciremos los requisitos funcionales y no funcionales en una representación del software.

Fases del desarrollo software.
Diseño

Podemos encontrar 4 tipos de diseño dentro de esta fase:



Fases del desarrollo software.
Diseño de datos

Se encarga de transformar el **modelo de dominio** de la información creado durante el análisis en las **estructuras de datos** que se utilizarán en la implementación.

Se pueden utilizar diagramas como entidad/relación o diagramas de clases UML, entre otros.

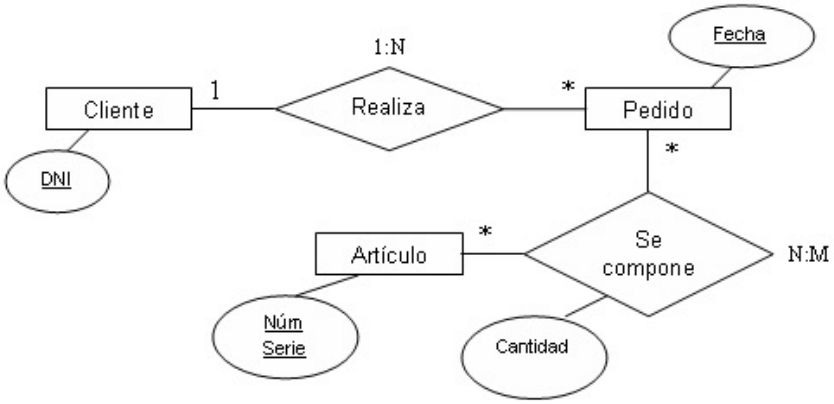


Diagrama entidad relación

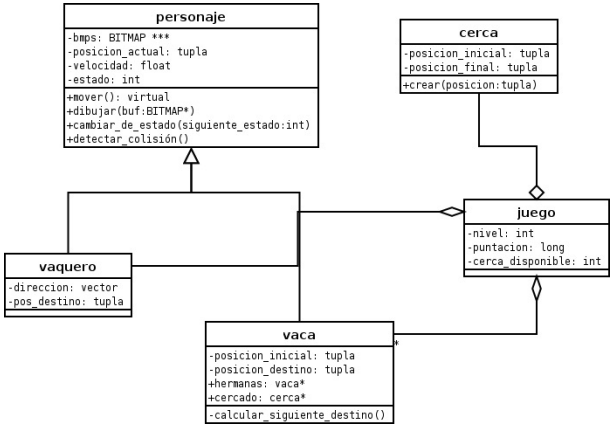


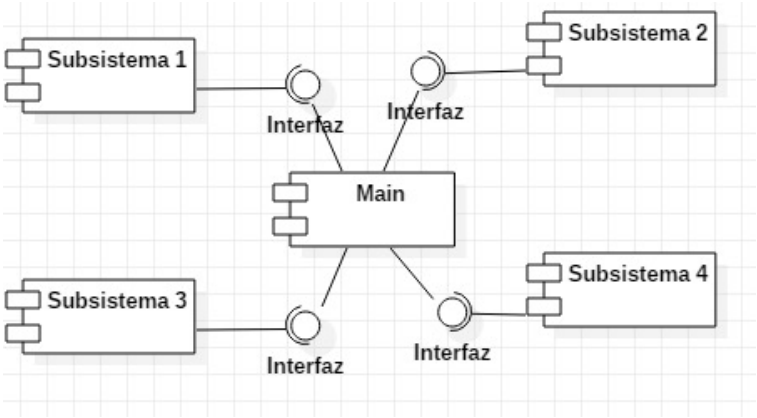
Diagrama de clases UML

Fases del desarrollo software.
Diseño arquitectónico

Se centra en la representación de la estructura de los componentes del software.

Un componente puede ser: un módulo de la aplicación, la base de datos, los conectores, etc.

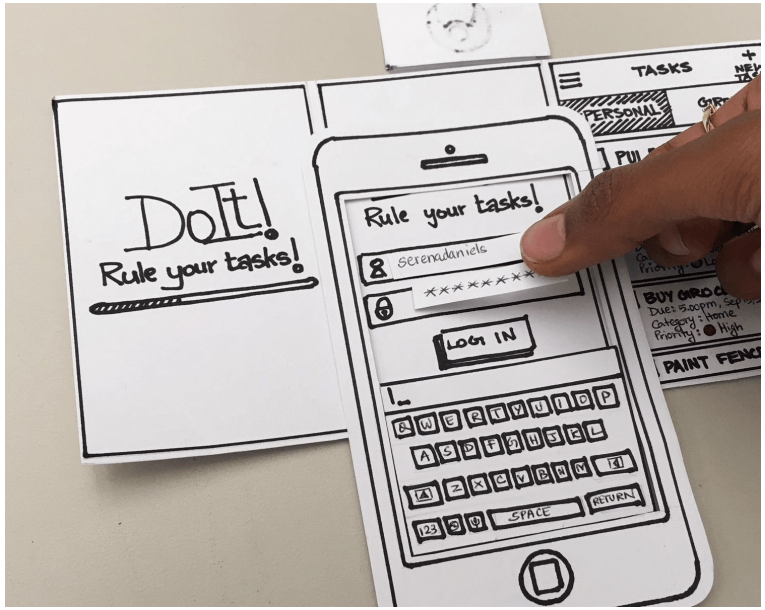
Se utilizarán diagramas que muestren la relación entre estos componentes, como por ejemplo los diagramas de componentes UML.



Fases del desarrollo software.
Diseño de la interfaz

Describe como se comunica el software consigo mismo, con los sistemas que operan con él, y con las personas que lo utilizan.

Para representar como pueden comunicarse las personas con el software, se pueden utilizar prototipos.



Ejemplo de prototipo a papel

Fases del desarrollo software.
Diseño procedimental

Transformar los elementos estructurales anteriormente diseñados en una descripción procedimental. Es decir, describir a gran nivel de detalle los algoritmos que implementarán los anteriores diseños.

El resultado de este diseño es un nivel de diseño suficiente para que sirva de guía a la hora de generar el código fuente.

Existen numerosas técnicas para realizar el diseño procedimental.

Fases del desarrollo software.
Diseño procedimental

1. Pseudocódigo: utiliza texto descriptivo para realizar el diseño del algoritmo.

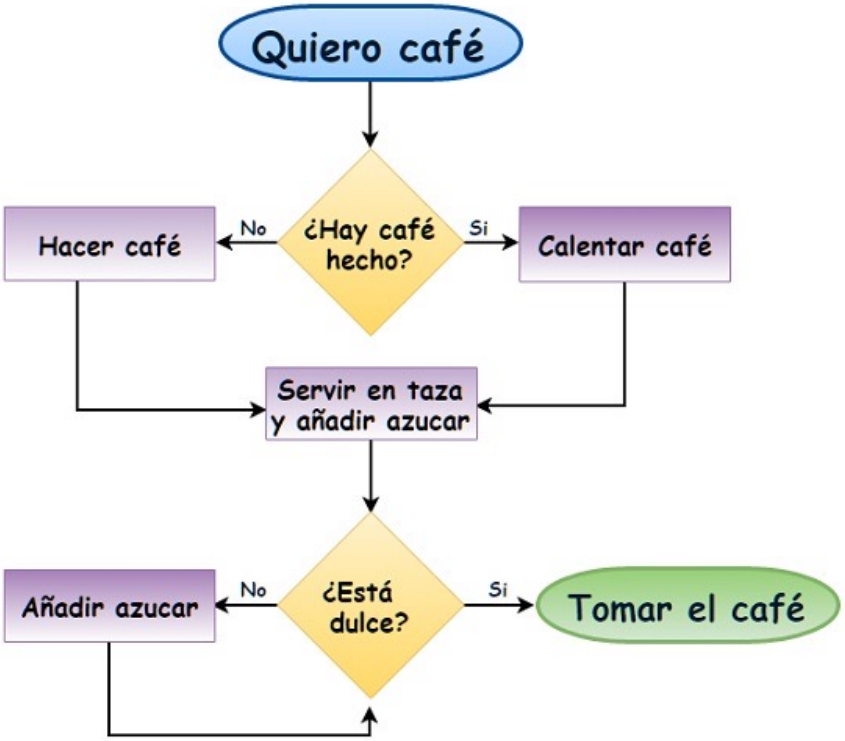
```
algoritmo Sumar

variables
  entero a, b, c

inicio
  escribir( "Introduzca el primer número (entero): " )
  leer( a )
  escribir( "Introduzca el segundo número (entero): " )
  leer( b )
  c ← a + b
  escribir( "La suma es: ", c )
fin
```

Fases del desarrollo software.
Diseño procedimental

2. Diagramas de flujo: se utilizan una serie de símbolos interconectados que tienen un significado concreto.



Fases del desarrollo software.
Diseño procedimental

3. Otros: diagrama de cajas, tablas de decisión, …

Implementación

Fases del desarrollo software.
Implementación

En base a lo establecido en la fase de diseño, el programador codifica el programa utilizando un lenguaje de programación concreto.

Para ello se basará en el diseño procedimental generado en la anterior fase.

Como resultado obtendremos el código fuente del programa.

Fases del desarrollo software.
Implementación

Cosas a tener en cuenta:

- Dependiendo del lenguaje de programación, podremos/tendremos que optar por un tipo de paradigma de programación.

- Se deben seguir unas guías de estilo concretas del lenguaje de programación o de la empresa.

- Dependiendo del lenguaje de programación, podremos/tendremos que utilizar un entorno de desarrollo concreto.

Pruebas

Fases del desarrollo software.
Pruebas

- Tradicionalmente, problemas de calidad:
  - Durante el proceso de desarrollo.
  - Con el producto entregado.

- Motivos:
  - Desviaciones en plazos y esfuerzo.
  - Fallos durante la implantación y mantenimiento.

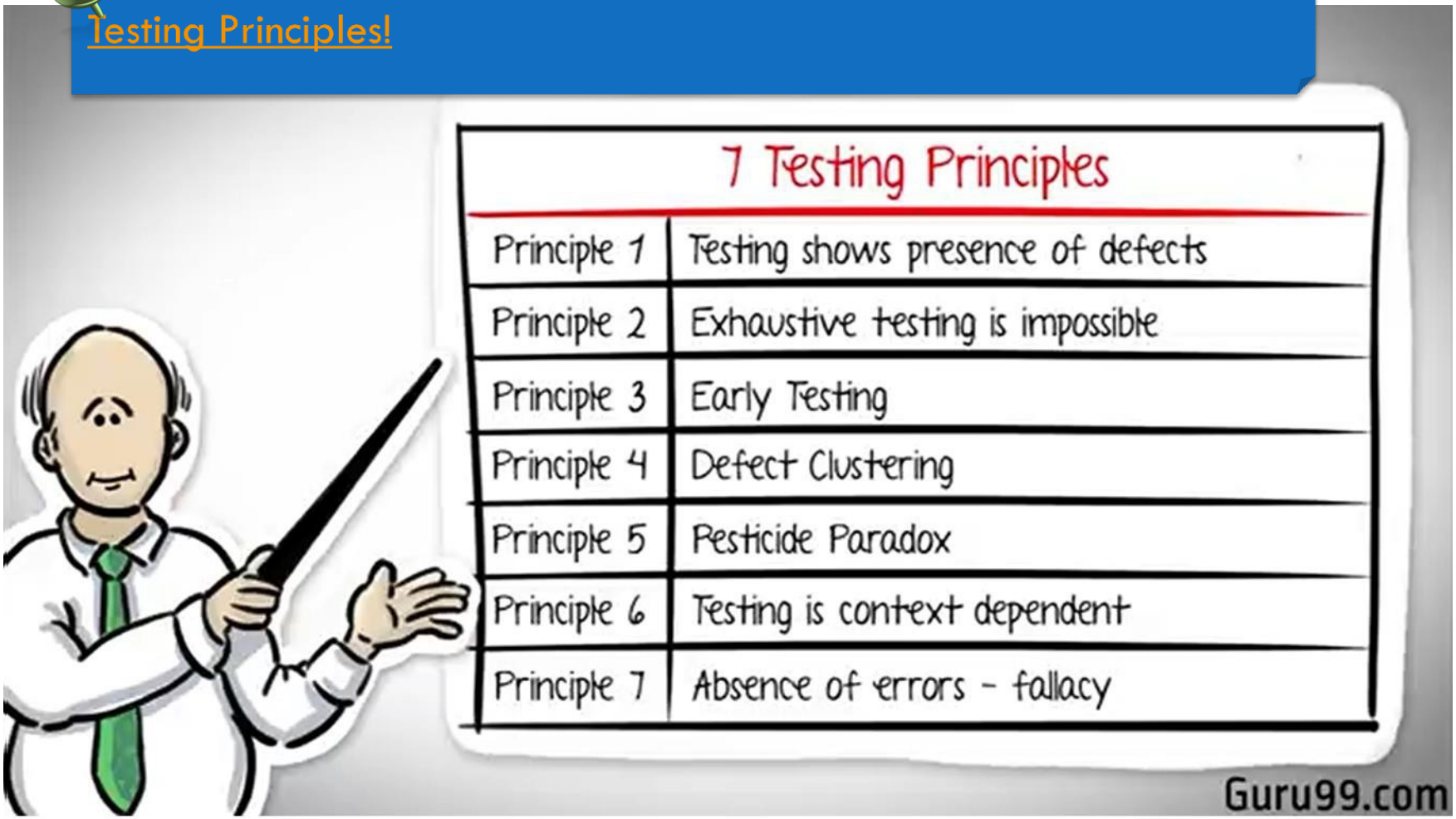
- No se le da la importancia suficiente a las **pruebas del software o *testing*.**
  - Habitual: disminuir plazo de pruebas para reducir plazos o costes.

Fases del desarrollo software.
Pruebas

- *Testing*:
  - Conjunto de procesos que permiten verificar y validar la calidad del SW indicando errores de diseño e implementación.
  - Se inicia según comienza el desarrollo y continua hasta el final del proyecto en sus diferentes fases.
  - Plan de prueba: detalla su planificación.
  - Entornos de prueba separados del de producción.
  - Habitual, que exista un equipo dedicado al testeo distinto a los desarrolladores → **lo cual no implica que los desarrolladores no deban probar!**

Fases del desarrollo software. **Pruebas**

Testing Principles!



https://es.myservername.com/7-principles-software-testing#1_Testing_Shows_the_Presence_of_Defects

Fases del desarrollo software.
Pruebas

- Testeo bajo condiciones controladas y conocidas:
  - Uso de diferentes herramientas, técnicas y métodos.
  - Las condiciones pueden ser normales o anormales, forzando al programa.
- Tipos de errores:
  - Bugs o errores de programación.
  - Defectos de forma o funcionales.

80% de los errores provienen de las primeras fases del proyecto.

El coste de corregir errores crece exponencialmente según avanza el proyecto.

Fases del desarrollo software.
Pruebas

- Principales motivos:
  - Falta de comunicación (cliente, diseñadores, analistas, programadores, etc.)
  - Sobre el SW: complejidad, poca reutilización.
  - Bugs: "ego del programador"
  - Cambios en requisitos.
  - Factor tiempo → reducir fases de pruebas.
  - Pobre documentación → dificulta modificaciones del código.

Fases del desarrollo software.
Pruebas

- Tipos de pruebas:
  - Verificación:
    - Comprobar el resultado de las distintas fases del desarrollo, según los parámetros de calidad que se definan.
    - ¿El producto está correctamente construido?
    - Herramientas: Ej.: *Check-lists* (listas de chequeo), reuniones formales dentro del equipo, etc.
  - Validación:
    - Comprobar los requisitos de software, lo que espera el usuario.
    - ¿El producto construido es correcto?
    - Comienza cuando la verificación está completa.

Fases del desarrollo software.
Pruebas

- Conceptos:
  - *Beta-testing*:
    - Probar una *versión beta* (anterior al producto final).
  - *Version alpha:*
    - Programa incompleto con cierta funcionalidad implementada.
  - *RTM testing:*
    - Se comprueba todas las funcionalidades del programa completo en producción.

Documentación

Fases del desarrollo software.
Documentación

- Información descriptiva del software.
- Se pueden ver las aplicaciones desde dos aspectos:
  - Descripción física o técnica:
  - Descripción funcional

C1.4: ¿Qué es el efecto Y2K?.

Explotación

Fases del desarrollo software.
Explotación

Fase en la que el cliente hace uso de la aplicación.

Mantenimiento

Fases del desarrollo software.
Mantenimiento

Durante la explotación del sistema, se puede ofrecer un soporte. Es el principal motor económico del software empresarial.

Roles en el ciclo de vida

Fases del desarrollo software.
Roles dentro del desarrollo

Analista de sistemas

- Fases: Análisis.
- Realiza un estudio de los requisitos y analiza el sistema en su conjunto.

Diseñador de software

- Fases: Diseño.
- Parte del análisis realizado y realiza el diseño.

Analista programador

- Fases: Desarrollo, Pruebas, Documentación, Explotación, Mantenimiento
- Tiene un conocimiento amplio tanto de la parte funcional como técnica, para apoyar en la tarea de codificación.

Programador

- Fases: Desarrollo, Pruebas, Documentación, Explotación, Mantenimiento
- Participa en la codificación del código fuente.

Arquitecto de software

- Fases: Análisis, Diseño, documentación y Explotación.
- Cohesiona todo el proceso de desarrollo.
- Se encarga de utilizar los mejores *frameworks (estructura conceptual y tecnológica de soporte definido)* y tecnologías según las necesidades y recursos disponibles.

Metodologías de desarrollo

Metodologías de desarrollo.

- Existen diferentes modelos para definir las etapas de desarrollo. Suelen combinarse.
  - Modelo en cascada.
  - Modelo en espiral.
  - Desarrollo iterativo e incremental.